과제 3가지 경로 테스트 (실습)

> 도구 미사용, 단일 도구, 복합 도구 3가지 질문을 직접 설계해 자신의 Agent가 모두 통과하는지 확인해봅니다.

오늘의 성공 기준은 도구 미사용, 단일 도구, 복합 도구 3가지 경로 통과입니다. 5번째 세션 5-3에서는 준비된 질문을 실행했지만 이번에는 질문을 직접 설계합니다. 좋은 질문을 설계하는 능력이 곧 Agent를 검증하는 능력입니다.

따라하기

1. 세 경로의 질문을 각각 1개 이상 직접 설계합니다. 통과 기준까지 함께 정합니다.
   - 도구 미사용 경로: 도구 없이 답해야 자연스러운 질문 (예: 인사와 역할 소개). 통과 기준은 messages에 tool_call이 하나도 없고 답이 자연스러운 것입니다.
   - 단일 도구 경로: 도구 하나로 답이 끝나는 질문. 통과 기준은 의도한 도구가 정확히 1회 호출되고 답이 도구 결과에 근거하는 것입니다.
   - 복합 도구 경로: 도구 2개 이상을 체이닝해야 하는 질문. 앞 도구의 결과가 뒤 도구의 인자에 필요하도록 설계하면 체이닝이 강제됩니다. 통과 기준은 필요한 도구가 모두 올바른 순서로 호출되는 것입니다.
2. 5번째 세션 5-3의 `run_and_trace`로 설계한 질문을 실행해 전체 메시지를 눈으로 확인합니다.
3. 실패하면 두 곳을 고친 뒤 재시도합니다. 첫째, 도구 docstring을 더 구체적으로 씁니다. 둘째, 시스템 프롬프트의 도구 사용 전략을 더 명확하게 씁니다. 어디를 고쳤더니 무엇이 달라졌는지 한 줄씩 기록해 두면 4일차의 좋은 도구의 조건과 바로 이어집니다.

4. 마지막 5분에는 오늘 설계한 검증 질문을 평가셋 파일 `eval_set.json`에 추가합니다. 스키마는 question, expected_source, type으로 고정입니다.

   2일차에 만든 `eval_set.json`을 `day3_practice`로 가져와 오늘 문항을 이어 추가합니다.

   그 파일이 없다면(2일차 미션을 건너뛰었다면) 2일차 6번째 세션의 예시 5문항으로 새로 만들어 오늘 문항을 추가합니다.

   이 파일은 매일 누적되어 7일차 평가 실습의 입력으로 쓰입니다.

```
[
  {"question": "김하늘 님은 어느 팀 소속인가요?", "expected_source": "search_employee", "type": "single_tool"},
  {"question": "'프린터가 고장 났어요' 문의 부서를 분류하고 48건이 12% 늘면 몇 건인지 계산해 줘.", "expected_source": "classify_request, calculate", "type": "multi_tool"},
  {"question": "어떤 일을 도와줄 수 있어?", "expected_source": "none", "type": "no_tool"}
]
```

막힐 때

- 복합 질문인데 모델이 도구 하나로 끝내 버립니다.

  앞 도구의 결과 없이는 뒤 도구의 인자를 정할 수 없는 구조인지 질문을 다시 봅니다. "A를 조회하고 B도 알려줘"는 병렬이라 체이닝이 아닐 수 있습니다. "A를 조회한 결과로 B를 계산해줘" 구조로 바꿉니다.

- 도구 미사용 질문인데 자꾸 도구를 호출합니다.

  시스템 프롬프트에 "도구가 필요 없는 질문에는 도구 없이 바로 답해"가 있는지 확인하고 질문이 도구 이름과 겹치는 단어를 쓰고 있지 않은지 봅니다.

- `ImportError: cannot import name 'create_react_agent'`가 납니다.

  최신 langgraph에서 이름이 `create_agent`로 바뀐 것입니다. `from langchain.agents import create_agent`로 바꿔 쓰되 프롬프트 인자 이름이 `prompt`가 아니라 `system_prompt`입니다. 5일차에서 이 API를 본격적으로 씁니다.

확인: 직접 설계한 3가지 경로 질문이 모두 통과하고 eval_set.json에 문항을 추가했으면 성공입니다.

> 검증 질문을 직접 설계해 Agent의 도구 선택을 3가지 경로로 확인하는 방법을 익혔습니다.

▼ 추가로 해볼 만한 것

- `graph.stream({"messages": [...]}, stream_mode="values")`로 바꿔 단계마다 State가 갱신되는 과정을 실시간으로 관찰해 봅니다.

- LangGraph에는 오늘 만든 구조를 한 줄로 만들어 주는 `create_react_agent` 프리빌트(미리 조립해 둔) 함수도 있습니다. `from langgraph.prebuilt import create_react_agent`로 같은 도구를 넣어 만들어 보고 오늘 직접 조립한 그래프와 동작을 비교해 봅니다. 직접 조립할 줄 알면 프리빌트가 안 맞을 때 열어서 고칠 수 있습니다.
- 3번째 세션 3-3의 반복 그래프에서 `graph.invoke(..., config={"recursion_limit": 4})`처럼 한도를 일부러 낮춰 `GraphRecursionError`를 재현해 봅니다. 에러 메시지를 읽어 두면 실무에서 무한 루프를 만났을 때 바로 알아볼 수 있습니다.

▼ 모범답안 (직접 해본 뒤 열어보세요)

세 경로로 설계한 질문입니다.

```
경로 1 (도구 미사용) : 너는 무슨 일을 도와줄 수 있어?
경로 2 (단일 도구)   : 정하은 님 이메일 좀 알려줘.
경로 3 (복합 도구)   : '사내 메일이 안 들어와요' 문의를 부서로 분류하고,
                       그 부서 이번 달 접수 32건이 25% 늘면 몇 건인지 계산해줘.
```

복합 경로의 trace입니다. 분류 결과가 나온 다음에 계산이 붙는 순서를 확인하면 됩니다.

```
============================================================
질문: '사내 메일이 안 들어와요' 문의를 부서로 분류하고, 그 부서 이번 달 접수 32건이 25% 늘면 몇 건인
지 계산해줘.
============================================================
  [HumanMessage] '사내 메일이 안 들어와요' 문의를 부서로 분류하고, ...
  [AIMessage] 도구 요청: classify_request({'text': '사내 메일이 안 들어와요'})
  [ToolMessage] IT지원
  [AIMessage] 도구 요청: calculate({'expression': '32 * 1.25'})
  [ToolMessage] 40.0
  [AIMessage] IT지원 담당입니다. 이번 달 32건이 25% 늘면 40건이 됩니다.
```

질문을 이렇게 짠 이유입니다.

- 복합 경로는 분류 결과가 나와야 계산 대상이 정해지도록 묶었습니다. "A 알려주고 B도 알려줘"로 쓰면 모델이 두 도구를 병렬로 부르고 끝나서 체이닝이 확인되지 않습니다.
- 도구 미사용 질문에는 조회나 계산처럼 도구 이름과 겹치는 단어를 넣지 않았습니다. 넣으면 모델이 괜히 도구를 부릅니다.

마지막 5분에 평가셋에 넣은 문항입니다.

```
[
  {"question": "정하은 님 이메일 좀 알려줘.", "expected_source": "search_employee", "type": "single_tool"},
  {"question": "'사내 메일이 안 들어와요' 문의를 부서로 분류하고 32건이 25% 늘면 몇 건인지 계산해줘.", "expected_source": "classify_request, calculate", "type": "multi_tool"},
  {"question": "너는 무슨 일을 도와줄 수 있어?", "expected_source": "none", "type": "no_tool"}
]
```
